

EXPERIMENT NO - 2

AIM

To implement CPU Scheduling Algorithms (FCFS, SJF, Round Robin) and compare their performance.

SOFTWARE REQUIRED

A Linux operating system

GCC (GNU Compiler Collection) for compiling C programs

Terminal or command line interface

THEORY

CPU scheduling is a process that allows one process to use the CPU while the execution of another process is on hold (in waiting state) due to unavailability of any resource like I/O etc., thereby making full use of CPU. The aim of CPU scheduling is to make the system efficient, fast, and fair.

- **First Come First Serve (FCFS):** The simplest scheduling algorithm that schedules processes according to their arrival times. It operates on a non-preemptive basis.
- **Shortest Job First (SJF):** Selects the waiting process with the smallest execution time to execute next. It minimizes the average waiting time.
- **Round Robin (RR):** Each process is assigned a fixed time slot (time quantum) in a cyclic way. It is designed especially for time-sharing systems.

PROCEDURE

1. Create a C Program that accepts the number of processes and their burst times.
2. Implement the logic for FCFS to calculate waiting time by adding the burst time of the previous process.
3. Implement SJF by sorting the processes based on their burst times in ascending order before calculating waiting and turnaround times.
4. Implement Round Robin by using a time quantum loop to incrementally process execution until completion, updating remaining burst times.
5. Calculate and display average waiting and turnaround times for performance comparison.

INPUT-

```
#include <stdio.h>
```

```
struct Process {  
    int pid; // Process ID  
    int at; // Arrival Time  
    int bt; // Burst Time  
    int ct; // Completion Time  
    int tat; // Turnaround Time  
    int wt; // Waiting Time  
    int rt; // Remaining Time (for RR/SJF)  
};
```

```
void display(struct Process p[], int n, char name[]) {  
    float avgWT = 0, avgTAT = 0;
```

```
    printf("\n--- %s ---\n", name);  
    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
```

```
    for (int i = 0; i < n; i++) {  
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",  
            p[i].pid,  
            p[i].at,  
            p[i].bt,  
            p[i].ct,  
            p[i].tat,  
            p[i].wt);
```

```
        avgWT += p[i].wt;  
        avgTAT += p[i].tat;
```

```
    }  
    printf("\nAverage WT = %.2f\n", avgWT / n);  
    printf("Average TAT = %.2f\n", avgTAT / n);
```

```
}
```

```

// FCFS Implementation
void fcfs(struct Process p[], int n) {
    int time = 0;
    for(int i = 0; i < n; i++) {
        if(time < p[i].at) {
            time = p[i].at;
        }
        time += p[i].bt;
        p[i].ct = time;
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;
    }
}

int main() {
    int n = 4;

    // Initialize processes: {pid, at, bt, ct, tat, wt, rt}
    struct Process p[4] = {
        {1, 0, 5, 0, 0, 0, 5},
        {2, 1, 3, 0, 0, 0, 3},
        {3, 2, 8, 0, 0, 0, 8},
        {4, 3, 6, 0, 0, 0, 6}
    };

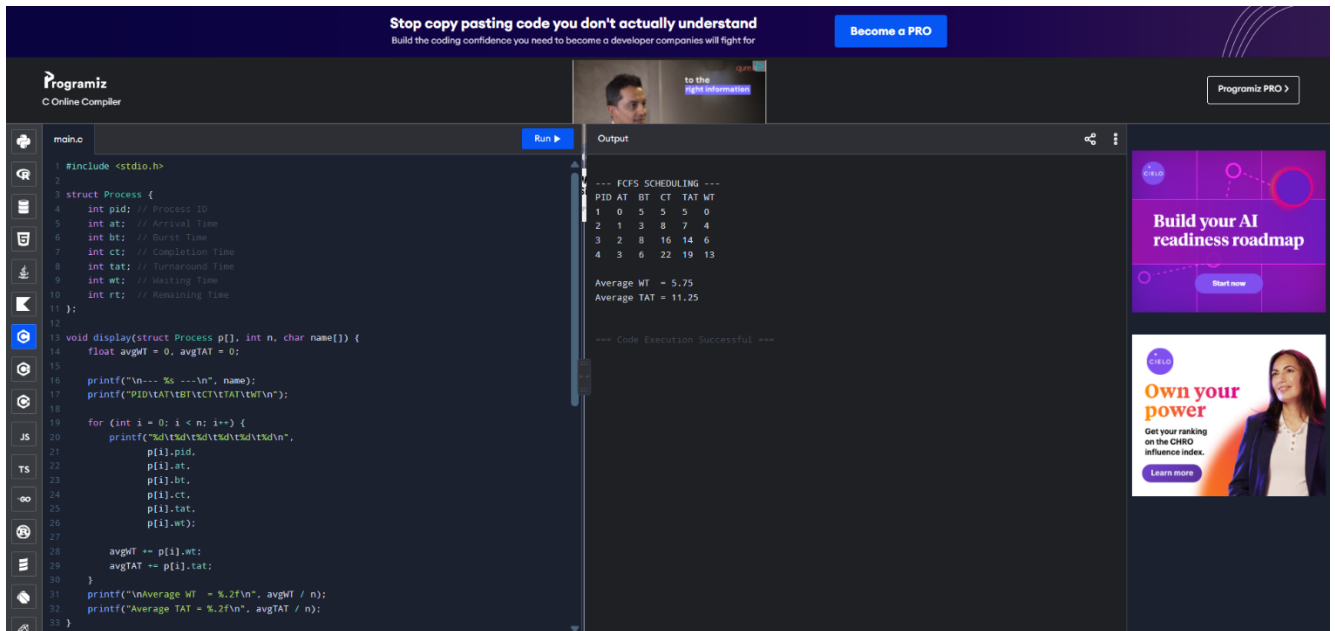
    fcfs(p, n);
    display(p, n, "FCFS SCHEDULING");

    // Further calls for SJF and RR would go here...

    return 0;
}

```

OUTPUT-



The screenshot shows the Programiz C Online Compiler interface. The code in the editor is as follows:

```
1 #include <stdio.h>
2
3 struct Process {
4     int pid; // Process ID
5     int at; // Arrival Time
6     int bt; // Burst Time
7     int ct; // Completion Time
8     int tat; // Turnaround Time
9     int wt; // Waiting Time
10    int rt; // Remaining Time
11 };
12
13 void display(struct Process p[], int n, char name[]) {
14     float avgWT = 0, avgTAT = 0;
15
16     printf("\n--- %s ---\n", name);
17     printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
18
19     for (int i = 0; i < n; i++) {
20         printf("%d\t%d\t%d\t%d\t%d\t%d\n",
21             p[i].pid,
22             p[i].at,
23             p[i].bt,
24             p[i].ct,
25             p[i].tat,
26             p[i].wt);
27
28         avgWT += p[i].wt;
29         avgTAT += p[i].tat;
30     }
31     printf("\nAverage WT = %.2f\n", avgWT / n);
32     printf("Average TAT = %.2f\n", avgTAT / n);
33 }
```

The output window displays the following results:

```
--- FCFS SCHEDULING ---
PID AT BT CT TAT WT
1 0 5 5 5 0
2 1 3 8 7 4
3 2 8 16 14 6
4 3 6 22 19 13

Average WT = 5.75
Average TAT = 11.25

--- Code Execution Successful ---
```

On the right side of the interface, there are two promotional banners. The top one is titled "Build your AI readiness roadmap" and the bottom one is titled "Own your power" with the text "Get your ranking on the CHRO Influence Index."

CONCLUSION

This experiment successfully demonstrated the implementation of CPU scheduling algorithms. FCFS was straightforward but suffered from higher average waiting times (convoy effect). SJF provided the optimal minimum average waiting time. Round Robin ensured fairness with a consistent response time across all processes, illustrating how different scheduling strategies impact overall system performance.